

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Microcontrollers

(CO3117)

Report Lab 5

Flow and Error Control in Communication

Advisor: TS. Lê Trọng Nhân

Student: Nguyễn Chí Thanh 2313078

Ho Chi Minh City, September 23, 2025

Mục lục

1 Overview	4
2 Introduction	4
3 Proteus simulation platform	6
4 Project configurations	7
4.1 UART Configuration	7
4.2 ADC Input	7
5 UART loop-back communication	9
6 Sensor reading	10
7 Project description	10
7.1 Command parser	11
7.2 Project implementation	13
7.2.1 Command parser FSM	13
7.2.2 UART communication FSM	14
7.2.3 While loop and UART	15

1 Overview

In this lab, we will explore various LED animations using a microcontroller. The goal is to understand how to control LEDs and create visually appealing effects. This report consists of:

- Source code C language for LED animations in STM32CubeIDE.
- Simulate LED animations using Proteus software.

For more details about source code, visit my [GitHub repository](#).

2 Introduction

Flow control and Error control are the two main responsibilities of the data link layer, which is a communication channel for node-to-node delivery of the data. The functions of the flow and error control are explained as follows.

Flow control mainly coordinates with the amount of data that can be sent before receiving an acknowledgment from the receiver and it is one of the major duties of the data link layer. For most of the communications, flow control is a set of procedures that mainly tells the sender how much data the sender can send before it must wait for an acknowledgment from the receiver.

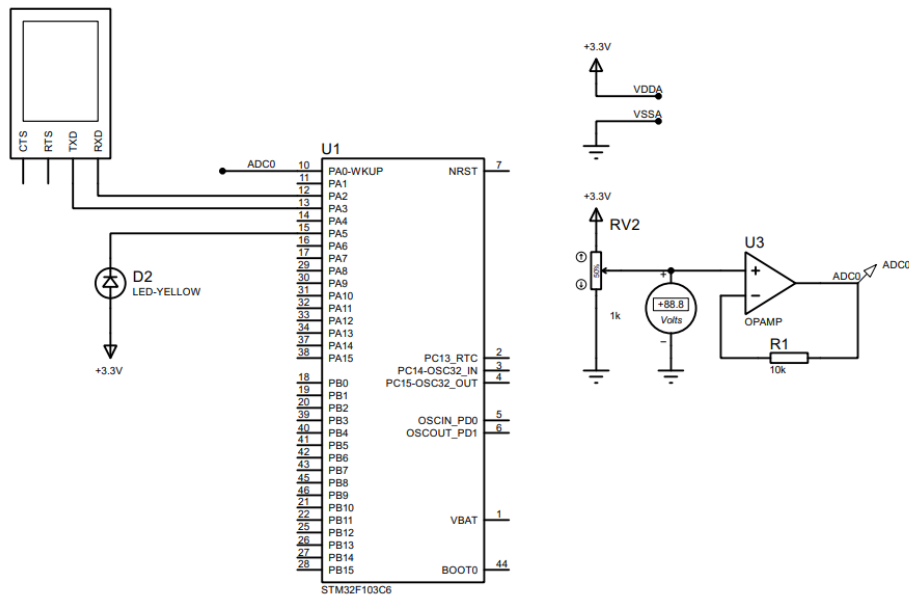
A critical issue, but not really frequently occurred, in the flow control is that the processing rate is slower than the transmission rate. Due to this reason each receiving device has a block of memory that is commonly known as buffer, that is used to store the incoming data until this data will be processed. In case the buffer begins to fill-up then the receiver must be able to tell the sender to halt the transmission until once again the receiver become able to receive.

Meanwhile, error control contains both error detection and error correction. It mainly allows the receiver to inform the sender about any damaged or lost frames during the transmission and then it coordinates with the re-transmission of those frames by the sender.

The term Error control in the communications mainly refers to the methods of error detection and re-transmission. Error control is mainly implemented in a simple way and that is whenever there is an error detected during the exchange, then specified frames are re-transmitted and this process is also referred to as Automatic Repeat request(ARQ).

The target in this lab is to implement a UART communication between the STM32 and a simulated terminal. A data request is sent from the terminal to the STM32. Afterward, computations are performed at the STM32 before a data packet is sent to the terminal. The terminal is supposed to reply an ACK to confirm the communication successfully or not.

3 Proteus simulation platform



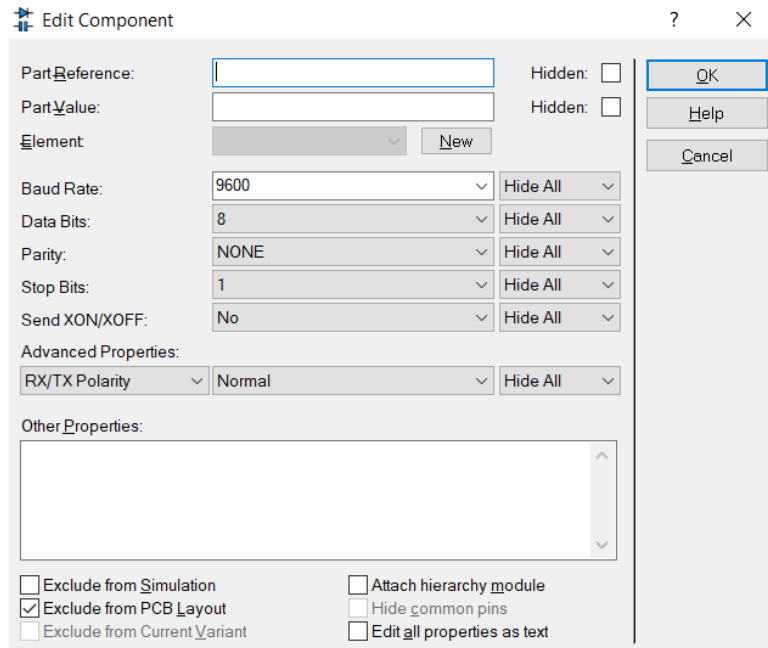
Hình 3.1: Simulation circuit on Proteus

Some new components are listed bellow:

- Terminal: Right click, choose Place, Virtual Instrument, then select VIRTUAL TERMINAL.
- Variable resistor (RV2): Right click, choose Place, From Library, and search for the POT-HG device. The value of this device is set to the default 1k.
- Volt meter (for debug): Right click, choose Place, Virtual Instrument, the select DC VOLTMETER.
- OPAMP (U3): Right click, choose Place, From Library, and search for the OPAMP device.

The opamp is used to design a voltage follower circuit, which is one of the most popular applications for opamp. In this case, it is used to design an adc input signal, which is connected to pin PA0 of the MCU.

Double click on the virtual terminal and set its baudrate to 9600, 8 data bits, no parity and 1 stop bit, as follows:



Hình 3.2: *Terminal configuration*

4 Project configurations

A new project is created with following configurations, concerning the UART for communications and ADC input for sensor reading. The pin PA5 should be an GPIO output, for LED blinky.

4.1 UART Configuration

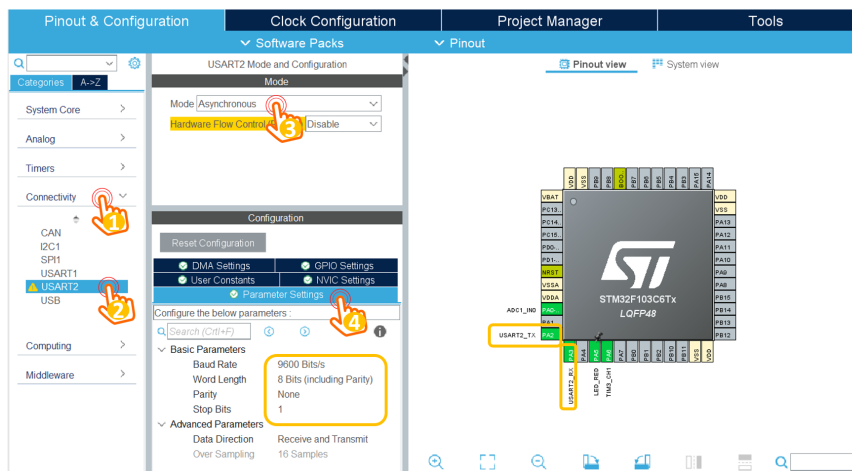
From the ioc file, select **Connectivity**, and then select the **USART2**. The parameter settings for UART channel 2 (USART2) module are depicted as follows:

The UART channel in this lab is the Asynchronous mode, 9600 bits/s with no Parity and 1 stop bit. After the uart is configured, the pins PA2 (Tx) and PA3(Rx) are enabled.

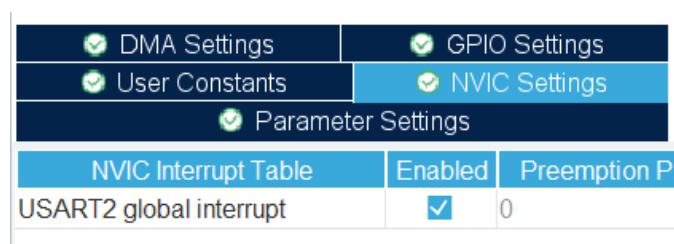
Finally, the NVIC settings are checked to enable the UART interrupt, as follows:

4.2 ADC Input

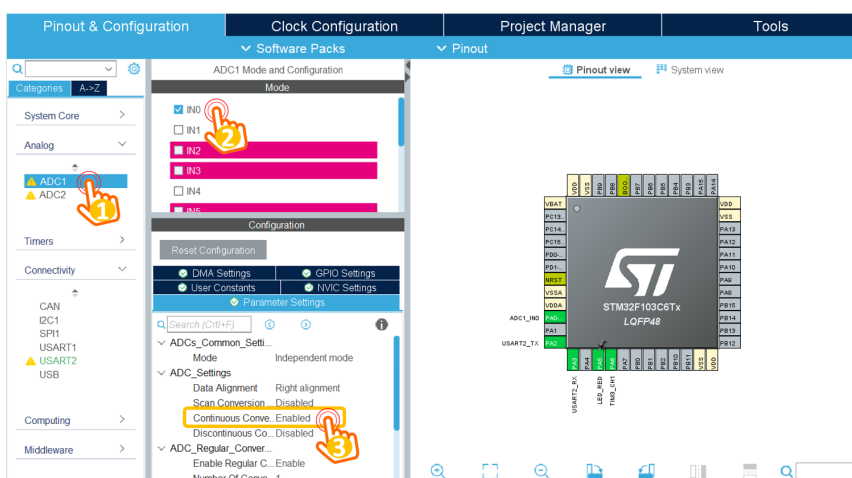
In order to read a voltage signal from a simulated sensor, this module is required. By selecting on **Analog**, then **ADC1**, following configurations are required:



Hình 4.1: UART configuration in STMCube



Hình 4.2: Enable UART interrupt



Hình 4.3: Enable UART interrupt

The ADC pin is configured to PA0 of the STM32, which is shown in the pinout view dialog.

Finally, the PA5 is configured as a GPIO output, connected to a blinky LED.

5 UART loop-back communication

This source is required to add in the main.c file, to verify the UART communication channel: sending back any character received from the terminal, which is well-known as the loop-back communication.

Listing 1: Implement the UART interrupt service routine

```
/* USER CODE BEGIN 0 */
uint8_t temp = 0;

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart){
    if(huart->Instance == USART2){
        HAL_UART_Transmit(&huart2, &temp, 1, 50);
        HAL_UART_Receive_IT(&huart2, &temp, 1);
    }
}
/* USER CODE END 0 */
```

When a character (or a byte) is received, this interrupt service routine is invoked. After the character is sent to the terminal, the interrupt is activated again. This source code should be placed in a user-defined section.

Finally, in the main function, the proposed source code is presented as follows:

Listing 2: Implement the main function

```
int main(void)
{
    HAL_Init();
    SystemClock_Config();

    MX_GPIO_Init();
    MX_USART2_UART_Init();
    MX_ADC1_Init();

    HAL_UART_Receive_IT(&huart2, &temp, 1);
}
```

```

while (1)
{
    HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
    HAL_Delay(500);
}
}

```

6 Sensor reading

A simple source code to read adc value from PA0 is presented as follows:

Listing 3: ADC reading from AN0

```

uint32_t ADC_value = 0;
while (1)
{
    HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
    ADC_value = HAL_ADC_GetValue(&hadc1);
    HAL_UART_Transmit(&huart2, (void *)str, sprintf(str, "%d\n", ADC_value), 1000);
    HAL_Delay(500);
}

```

Every half of second, the ADC value is read and its value is sent to the console. It is worth noticing that the number ADC_value is convert to ascii character by using the sprintf function.

The default ADC in STM32 is 13 bits, meaning that 5V is converted to 4096 decimal value. If the input is 2.5V, ADC_value is 2048.

7 Project description

In this lab, a simple communication protocol is implemented as follows:

- From the console, user types **!RST#** to ask for a sensory data.
- The STM32 response the ADC_value, following a format **!ADC=1234#**, where 1234 presents for the value of ADC_value variable.

- The user ends the communication by sending **!OK#**

The timeout for waiting the **!OK#** at STM32 is 3 seconds. After this period, its packet is sent again. **The value is kept as the previous packet.**

7.1 Command parser

This module is used to received a command from the console. As the reception process is implement by an interrupt, the complexity is considered seriously. The proposed implementation is given as follows.

Firstly, the received character is added into a buffer, and a flag is set to indicate that there is a new data.

Listing 4: Add the received character into a buffer

```
#define MAX_BUFFER_SIZE 30
uint8_t temp = 0;
uint8_t buffer[MAX_BUFFER_SIZE];
uint8_t index_buffer = 0;
uint8_t buffer_flag = 0;
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart){
    if(huart->Instance == USART2){

        //HAL_UART_Transmit(&huart2, &temp, 1, 50);
        buffer[index_buffer++] = temp;
        if(index_buffer == 30) index_buffer = 0;

        buffer_flag = 1;
        HAL_UART_Receive_IT(&huart2, &temp, 1);
    }
}
```

A state machine to extract a command is implemented in the while(1) of the main function, as follows:

Listing 5: State machine to extract the command

```
while (1){
    if(buffer_flag == 1){
        command_parser_fsm();
    }
}
```

```
    buffer_flag = 0;
}
}
```

The output of the command parser is to set **command_flag** and **command_data**. In this project, there are two commands, **RTS** and **OK**. The program skeleton is proposed as follows:

Listing 6: Program structure

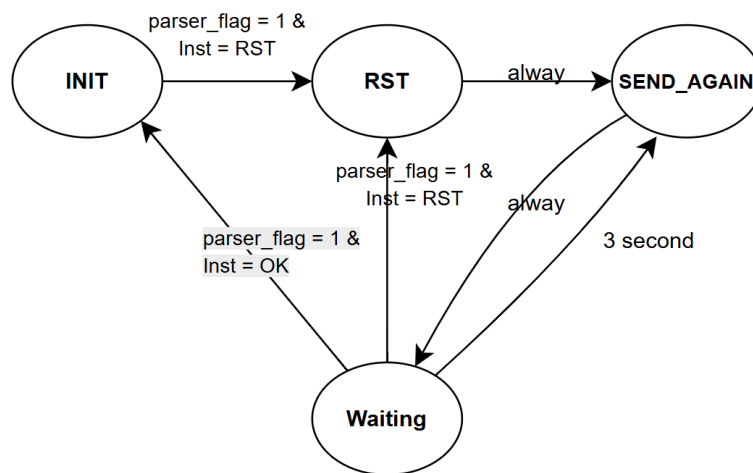
```
while (1){
    if(buffer_flag == 1){
        command_parser_fsm();
        buffer_flag = 0;
    }
    uart_communiation_fsm();
}
```

7.2 Project implementation

Students are proposed to implement 2 FSM in separated modules. Students are asked to design the FSM before their implementations in STM32Cube.

7.2.1 Command parser FSM

The command parser FSM is designed as follows:



Hình 7.1: *FSM for command parser*

The program code is implemented as follows:

Listing 7: Program code for command parser FSM

```
void command_parser_fsm(){
    if(SendData == 3){
        if(Inst == 2){
            Inst = 0;
            SendData = 0;
            return;
        }
        if(Inst == 1){
            Inst = 0;
            SendData = 1;
        }
        else if(istimer_flag(1) == 1){
            SendData = 2;
        }
    }
}
```

```

    }
}
if(SendData == 0){
    if(Inst != 1) return;
    else{
        Inst = 0;
        SendData = 1;
    }
}
if(SendData == 1){
    ADC_value = HAL_ADC_GetValue(&hadc1);
    len = sprintf(str, "!ADC=%lu#\r\n", ADC_value);
    SendData = 2;
}
if(SendData == 2){
    HAL_UART_Transmit(&huart2, (void*)str, len, 1000);
    SendData = 3;
    setTimer(1, 3000);
}
return;
}

```

With SendData = 0 is INIT state, SendData = 1 is READ_SENSOR state, SendData = 2 is SEND_DATA state and SendData = 3 is WAIT_ACK state.

7.2.2 UART communication FSM

This implementation is not use FSM to extract the command from the buffer. Instead, it checks the last characters in the buffer to find out if there is a valid command. The program skeleton is proposed as follows:

Listing 8: Program structure for UART communication FSM

```

void uart_communication_fsm(){
    if (index_buffer < 4) return;
    if (buffer[index_buffer - 1] == '#') {
        if (index_buffer >= 5 &&
            buffer[index_buffer - 5] == '!' &&
            buffer[index_buffer - 4] == 'R' &&
            buffer[index_buffer - 3] == 'S' &&

```

```

        buffer[index_buffer - 2] == 'T') {
            parser_flag = 1;
            Inst = 1;
            index_buffer = 0;
        }
        else if (index_buffer >= 4 &&
            buffer[index_buffer - 4] == '!' &&
            buffer[index_buffer - 3] == '0' &&
            buffer[index_buffer - 2] == 'K') {
            parser_flag = 1;
            Inst = 2;
            index_buffer = 0;
        }
    }
}

```

with **Inst = 1** is instruction **!RST#** and **Inst = 2** is instruction **!OK#**.

7.2.3 While loop and UART

Finally, the main while loop is implemented as follows:

Listing 9: Program structure

```

setTimer(0, 70);

while (1)
{
    /* USER CODE END WHILE */
    /* USER CODE BEGIN 3 */
    if(SendData == 3){
        if(istimer_flag(0) == 1){
            HAL_GPIO_TogglePin(LED_PA5_GPIO_Port, LED_PA5_Pin);
            setTimer(0, 500);
        }
    }
    if(parser_flag == 1 || SendData == 3){
        command_parser_fsm();
        parser_flag = 0;
    }
}

```

```
    uart_communication_fsm();  
}
```

Listing 10: Program for UARTRx complete callback

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef * huart){  
    if( huart->Instance == USART2 ) {  
        buffer[index_buffer++] = temp ;  
        if(index_buffer==30) index_buffer = 0;;  
        HAL_UART_Receive_IT(&huart2, &temp, 1);  
    }  
}
```

References